

Relatório de Decisões de Implementação

Sistema de Autocomplete de Jogos com Trie — C++

1 Representação da Trie

1.1 Alfabeto e tamanho

A Trie utiliza um alfabeto de **36 caracteres**: os dígitos '0'-'9' (índices 0–9) e as letras 'a'-'z' / 'A'-'Z' (índices 10–35). Cada nó armazena um array de 36 ponteiros para filhos (`ALPHABET_SIZE = 36`), um por caractere válido. Espaços são descartados durante a conversão e não ocupam posição no alfabeto.

1.2 Chave de busca (`toSearchKey`)

Antes de qualquer inserção ou busca, o título ou prefixo é convertido para uma chave interna pelo método `toSearchKey`. O método percorre cada caractere do texto e aplica o mapeamento:

```
'0'-'9'    ->  indices 0-9
'a'-'z' e 'A'-'Z' ->  indices 10-35
espacos    ->  ignorados
```

O resultado é uma string cujos bytes são valores no intervalo 0–35, usados diretamente como índices no array de filhos de cada nó. Essa representação garante que a busca seja *case-insensitive* e ignore espaços: "Half Life", "halflife" e "HALF LIFE" produzem a mesma chave.

2 Funcionamento dos Métodos

2.1 `insert`

Converte o título do jogo para chave de busca e percorre a Trie caractere a caractere, criando novos `TrieNodes` conforme necessário. No último nó do caminho, define `isEndOfTitle = true` e armazena o ponteiro para o objeto `Game` já existente no array da base — sem criar cópias dinâmicas.

2.2 `contains`

Converte o título para chave de busca e navega pelos nós correspondentes. Retorna `true` somente se o caminho existe integralmente na Trie e o nó final tem `isEndOfTitle == true`.

2.3 `autocomplete`

Converte o prefixo para chave de busca e navega até o nó correspondente. A partir desse nó, o método auxiliar `depthSearchGames` realiza uma busca em profundidade (DFS) recursiva, coletando todos os jogos cujos títulos compartilham aquele prefixo. O vetor resultante é ordenado por `sortResults` e truncado para os k primeiros elementos. Se o prefixo não existe na Trie ou $k \leq 0$, retorna um vetor vazio.

2.4 `sortResults`

Implementa selection sort sobre o vetor de ponteiros. O critério de ordenação é: maior popularidade primeiro; em caso de empate, ordem alfabética pela chave de busca do título (comparação lexicográfica sobre os índices 0–35).

3 Custo Computacional das Operações

Operação	Complexidade	Justificativa
<code>insert</code>	$O(L)$	Percorre L nós, onde L é o comprimento da chave de busca do título.
<code>contains</code>	$O(L)$	Navegação direta pelos L nós do caminho; sem backtracking.
<code>autocomplete</code>	$O(p + s + r^2)$	$O(p)$ para navegar ao prefixo; $O(s)$ para a DFS na subárvore; $O(r^2)$ para o selection sort sobre os r jogos encontrados.
<code>sortResults</code>	$O(r^2)$	Selection sort com dois laços aninhados sobre os r resultados.
<code>depthSearchGames</code>	$O(s)$	Visita exatamente os s nós da subárvore do prefixo.

Onde L é o comprimento do título após conversão para chave de busca, p é o comprimento do prefixo, s é o número de nós visitados na subárvore e r é o número de jogos encontrados para o prefixo buscado.

4 Comparação com Solução Ingênua e Uso de Memória

4.1 Comparação com varredura linear

Uma solução ingênua percorreria todos os N jogos do catálogo a cada chamada de `autocomplete`, verificando se cada título começa com o prefixo — custo $O(N \times L)$. Com a Trie, a busca navega diretamente ao nó do prefixo em $O(p)$ e explora apenas a subárvore relevante em $O(s)$, onde $s \leq N \times L$. Para catálogos grandes e prefixos longos, a vantagem da Trie é significativa.

4.2 Custo de memória

Cada `TrieNode` aloca um array fixo de 36 ponteiros, independente de quantos filhos realmente existem. Para um catálogo esparsos, isso representa desperdício: um nó com apenas um filho ocupa espaço para 35 ponteiros nulos. O consumo total de memória é proporcional ao número de nós criados, que no pior caso é $O(N \times L)$ — um nó por caractere de cada título sem prefixos compartilhados.

4.3 Compartilhamento de prefixos

A principal vantagem de memória da Trie aparece quando títulos compartilham prefixos. *Hades*, *Halo*, *Half Life* e *Harvest Moon* compartilham os nós de 'h' e 'a', criados uma única vez. Catálogos com muitos títulos de mesma franquia ou gênero se beneficiam especialmente desse compartilhamento.

4.4 Impacto do tamanho do alfabeto

Com `ALPHABET_SIZE = 36`, cada nó ocupa $36 \times \text{sizeof}(\text{ponteiro})$ bytes no array de filhos (288 bytes em sistemas 64-bit, apenas para os ponteiros). Alfabetos menores reduziriam esse custo, mas exigiriam tratamento adicional de caracteres. Para o escopo deste trabalho — títulos com apenas letras e dígitos — o alfabeto de 36 é o mínimo necessário.